Transfer Learning & the DocScan Capstone

Week 16 — Standing on a giant's shoulders, and shipping the project

Practical Image Processing

Lecture 16 ▪ 60–70 min

The finale

- Week 12: HOG+SVM plateaued at **0.48** on look-alike cars.
- Week 13: CNNs learn features — but need lots of data.
- Today: **transfer learning** — reuse a net trained on millions of images — and assemble the whole **DocScan** pipeline.

Today's real result

Same cars, same SVM, but **deep features** from a pretrained net: **0.48** $\rightarrow$ **0.87**.

Roadmap

The small-data problem

Transfer learning

The DocScan capstone

In-class exercise

Course wrap-up

The small-data problem

Why we can't just train a big CNN

- Deep CNNs have millions of parameters and need 10^4–10^6 labelled images.
- We have 60 images per car model — training a big net from scratch would badly overfit.
- Yet the HOG ceiling (0.48) shows hand features aren't enough. What now?

Key idea

Most real projects have *small* data. Transfer learning is how you still use deep learning.

The key insight

- A CNN trained on ImageNet (1.2M images, 1000 classes) learned *general* visual features: edges, textures, shapes, parts.
- Those early/mid features are useful for *almost any* image task — including cars.
- So: **reuse** those learned features instead of learning them from scratch.

Key idea

The first layers of a good vision net are a general-purpose feature extractor. Don't waste them.

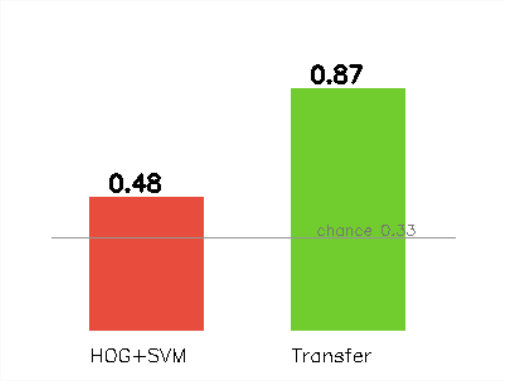
Transfer learning

Two flavours of transfer learning

Approach	What you do	When
Feature extraction	freeze the net, train a new classifier head	very small data
Fine-tuning	unfreeze some top layers, train them too	more data

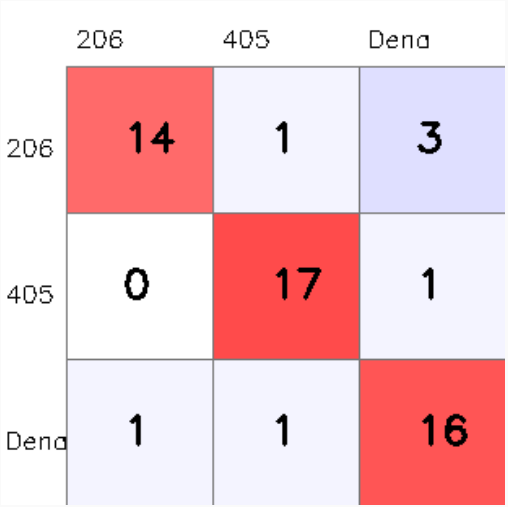
- We use **feature extraction**: a frozen ImageNet ConvNeXt turns each car into a 1024-D vector, then an SVM classifies — exactly Week 12's classifier, better features.

Our result: the ceiling lifts



- **Same** 180 images, **same** SVM, **same** split.
- Only change: HOG $\rightarrow$ frozen ImageNet features.
- 0.48 $\rightarrow$ **0.87**. The features were the bottleneck all along.

Confusion matrix after transfer



- The diagonal is now dominant — most cars classified correctly.
- Compare to Week 12's near-random confusion matrix.
- The pretrained net "sees" the fine differences HOG missed.

Why this works so well

- ImageNet includes many vehicles — the net already learned car-relevant parts.
- Its mid-level features (grille shapes, light patterns) are exactly what separates models.
- We get the power of a huge net with only 60 images per class and seconds of training.

✓ Good practice

Transfer learning is the single most useful deep-learning technique for real, small-data projects.

What each layer transfers

- **Early layers**: edges, colours, textures — universal, transfer to *any* task.
- **Middle layers**: parts and motifs — transfer well to related domains.
- **Late layers**: class-specific ("golden retriever") — least transferable; often replaced.

Key idea

The more your task differs from ImageNet, the more late layers you should retrain.

The domain gap

- ImageNet is natural photos. If your images are very different (X-rays, thermal, tiny digits), the transfer is weaker.
- Meters and cars are ordinary photos $\rightarrow$ the gap is small $\rightarrow$ transfer works great (our 0.87).
- Larger gap $\rightarrow$ fine-tune more layers, or find a model pretrained on closer data.

Math

Rule of thumb: small gap + small data $\rightarrow$ freeze & extract; large gap + more data $\rightarrow$ fine-tune.

Data augmentation for small data

- Multiply your data for free: random flips, small rotations, brightness/contrast jitter, crops.
- Each variant teaches the same label under a new nuisance $\rightarrow$ more invariance, less overfitting.
- Especially valuable when combined with transfer learning on tiny datasets.

✓ Good practice

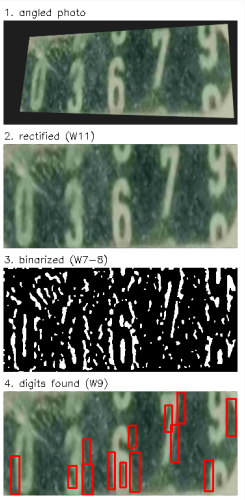
Augmentation is the cheapest accuracy you will ever buy — always try it on small-data problems.

Practical transfer-learning tips

- Use the pretrained model's *own* preprocessing (resize, normalize) — mismatch hurts a lot.
- Start with **feature extraction**; only fine-tune if you have enough data and need more.
- Freeze early layers (general), fine-tune late layers (task-specific) if you do fine-tune.
- Watch for overfitting — small data still bites; use augmentation and regularization.

The DocScan capstone

DocScan: the whole course in one pipeline



- **1. Photo** — a meter snapped at an angle (raw input).
- **2. Rectify** (W11) — homography warps it flat.
- **3. Binarize** (W2/7/8) — threshold + morphology clean it.
- **4. Segment** (W9) — contours find each digit box.
- **5. Read** (W13) — a CNN classifies each digit → “03679”.

Every week was a stage

Weeks	DocScan role
1–4 Pixels, histograms, filtering, denoise	clean the raw photo
5–6 Edges, Fourier	find structure, remove periodic noise
7–9 Morphology, segmentation, contours	isolate the digits
10–11 Features, homography	rectify to a flat scan
12–16 ML, CNNs, transfer learning	read the digits

Key idea

DocScan isn't a separate project — it's the spine that every week added a vertebra to.

Engineering a robust pipeline

- **Confidence gates**: if RANSAC inliers are too few, ask for a new photo (don't guess).
- **Sanity checks**: a meter reading has a fixed number of digits — reject wrong counts.
- **Fallbacks**: if the feature path fails, try the contour path (and vice-versa).
- **Logging**: record per-stage quality (RMSE, digit confidences) to debug failures.

✕ Pitfall

A demo works on one clean image; a product survives blur, glare, tilt, and dirt. Design for failure.

In-class exercise

In-class exercise (10 min)

Design decisions

1. You have 40 images per class. From-scratch CNN or transfer learning? Why?
2. Transfer learning lifted cars 0.48→0.87 with the *same* SVM. What changed, exactly?
3. In DocScan, the rectify step returns very few RANSAC inliers. What should the system do?

Debrief

- (1) **Transfer learning** — 40 images is far too few to train a deep net from scratch without overfitting.
- (2) Only the **features**: hand-designed HOG → pretrained deep features. The classifier was identical.
- (3) **Gate it**: treat the rectification as unreliable and prompt for a new photo rather than reading a bad scan.

Key idea

Good ML engineering is mostly good *features* and honest *failure handling* — not fancier models.

Course wrap-up

The arc of the course

- We started with a single pixel and ended with a pretrained neural network.
- Classical IP (weeks 1–11): precise, interpretable, no training — still essential for geometry & cleanup.
- Learning (weeks 12–16): powerful for recognition, especially via transfer learning.
- A real system uses both: classical to prepare, learned to recognize.

Choosing the right tool

If you need to...	Reach for...
align/rectify/measure geometry	classical (homography, contours)
clean noise, threshold, morphology	classical filters
recognize with small data	transfer learning
recognize with big data	train/fine-tune a CNN

Key idea

The mark of an engineer: picking the simplest tool that solves the problem reliably.

Honesty, ethics, and limits

- Report the *test* number, the failure cases, and the data you trained on — not just the best demo.
- A misread meter digit is a wrong bill; know your error rate and gate low-confidence reads.
- Beware dataset bias: if all training meters are clean and bright, expect failure on dirty ones.

✕ Pitfall

"97% accurate" means nothing without: on what data, which metric, and how it fails.

A quick self-check on the whole course

- Can you clean a noisy image (denoise, threshold, morphology)? (W1–8)
- Can you find and measure shapes/contours? (W9)
- Can you rectify a tilted photo with a homography? (W10–11)
- Can you build a features→classifier and a CNN, and use transfer learning? (W12–16)

Key idea

If yes to all four, you can build the majority of practical vision systems end to end.

Where to go next

- **Detection** (YOLO, Faster-RCNN): find *and* locate objects — the next step past classification.
- **Segmentation** (U-Net, SAM): label every pixel — for your power-line/mask datasets.
- **OCR** systems (Tesseract, TrOCR): full text reading pipelines built on these ideas.
- **Deployment**: on-device (TFLite, ONNX), latency, and robustness in the field.

The capstone deliverable

Final project (see `capstone/`)

1. Assemble the full DocScan pipeline on real `E:/data` images.
2. Rectify (W11) $\rightarrow$ clean/segment (W7–9) $\rightarrow$ read digits (W13 or transfer).
3. Add at least two robustness features (confidence gate, digit-count check, fallback).
4. Report end-to-end accuracy and where it fails; propose one concrete improvement.

From one pixel to a working scanner.

Classical vision to prepare, deep learning to recognize — and the judgment to know which, when.